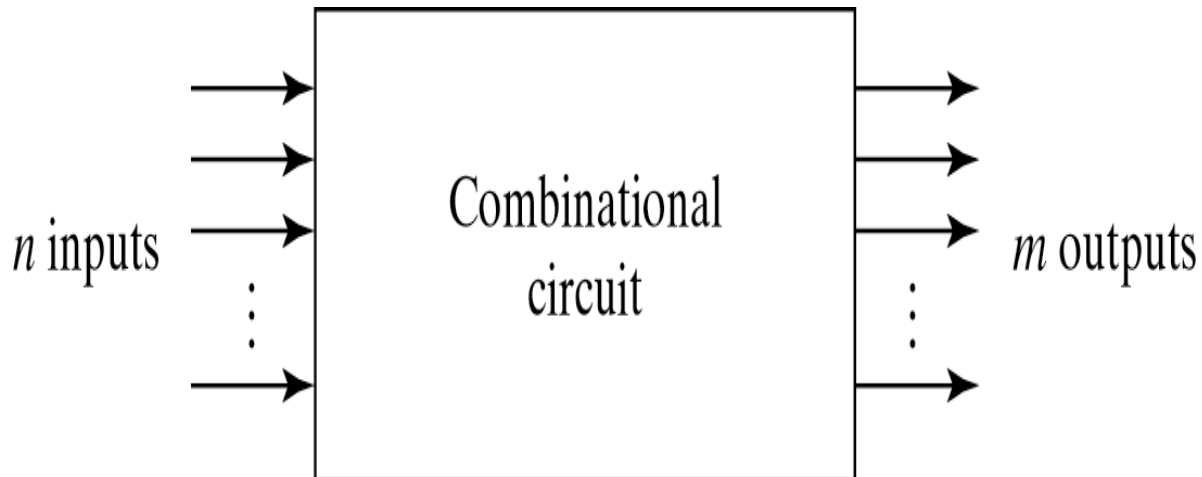


COMBINATIONAL CIRCUITS

A logic circuit whose output depends directly on only inputs is known as combinational circuit.

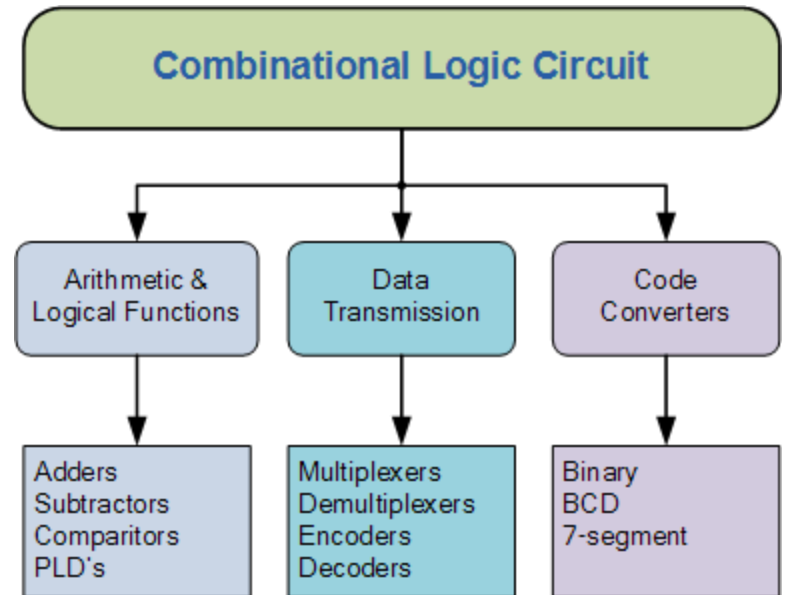
Combinational Logic Circuits are made up from basic logic NAND, NOR or NOT gates that are “combined” or connected together to produce more complicated switching circuits.



Classification of Combinational Logic

The function of a combinational logic circuit are:

1. Boolean Algebra – This forms the algebraic expression showing the operation of the logic circuit for each input variable
2. Truth Table – A truth table defines the function of a logic gate by providing a concise list that shows all the output states in tabular form for each possible combination of input variable.
3. Logic Diagram – This is a graphical representation of a logic circuit that shows the wiring and connections of each individual logic gate, represented by a specific graphical symbol, that implements the logic circuit.



Arithmetic Circuits

Arithmetic Circuits perform mathematical functions like Addition, Subtraction and Multiplication.

Problem Definition

- We will start out simply, adding two single digit binary numbers
- Let's make certain we can do this by hand before we construct the circuit
- There are four possible combinations of single digit, two number addition:
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 10$

Notice that one of the combinations of two single digit additions is a double digit number, created by a carry:

$$1 + 1 = 10$$

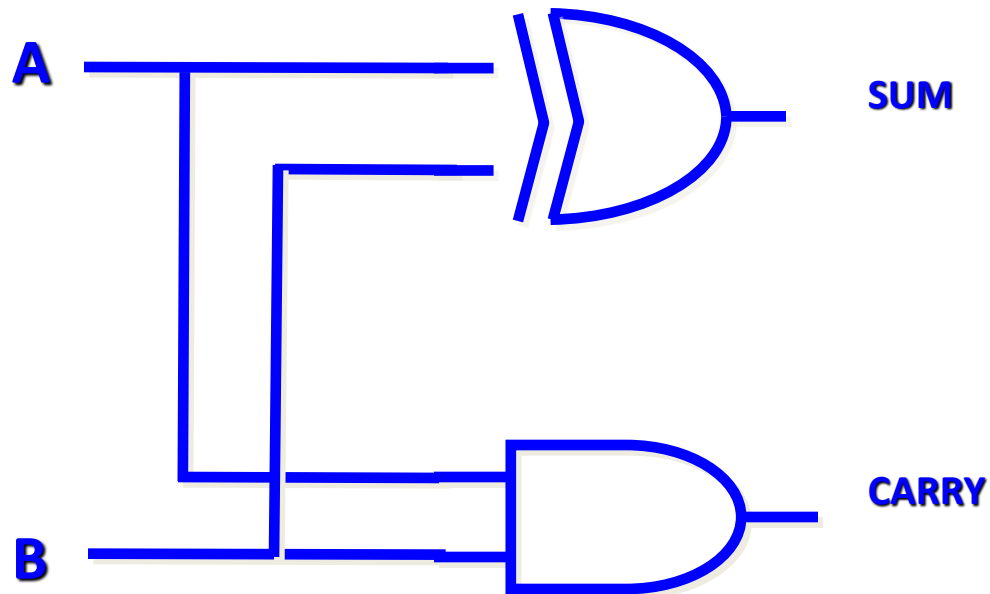
Truth Table for Adding Two 1-Bit Numbers

INPUTS		OUTPUTS	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

INPUTS		OUTPUTS	
A	B	Carry	Sum
		$A \cdot B$	$A \oplus B$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- The XOR circuit delivers the *Sum* of adding two single digits.
- The AND circuit produces the *Carry*.
- We've just created a fundamental circuit called a *Half Adder*, which provides the capability of adding two single bit numbers.

Circuit Diagram for the Half Adder

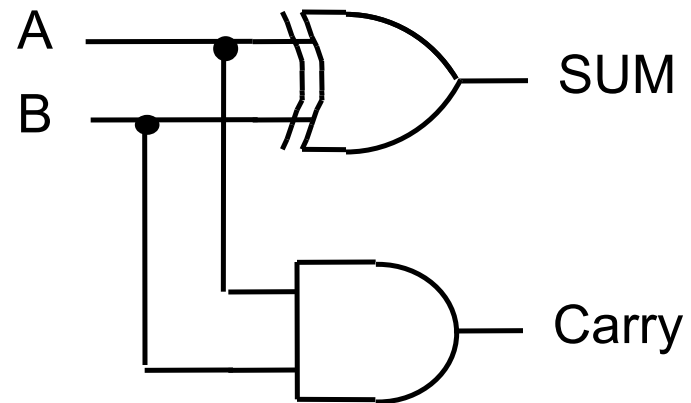


Half Adder

Add two binary numbers

- A, B -> single bit inputs
- Sum -> single bit sum
- Carry -> carry out

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Dec	Binary
1	1
+1	+1
2	10

Full Adder

Full Adder is a combinational circuit that performs the addition of three bits (two significant bits and previous carry).

The full adder is usually a component in a cascade of adders, which add 8, 16, etc, binary numbers.

C_i	A_i	B_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$C_i \backslash A_i B_i$	$A_i B_i$			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

Full Adder

Now consider implementation of carry out

Minimize circuit for carry out - C_{i+1}

C_i	A_i	B_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$A_i B_i$		00	01	11	10
C_i	0			1	
	1		1	1	1

C_{i+1}

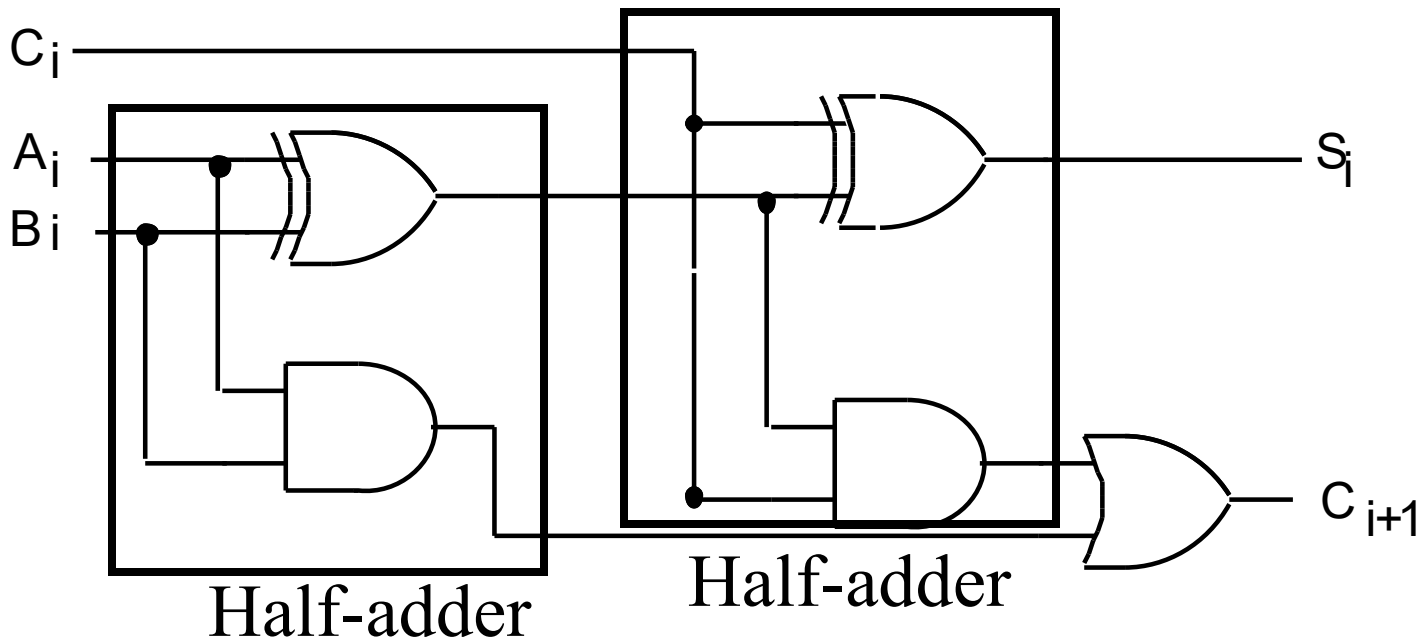
$$C_{i+1} = A_i B_i + C_i B_i + C_i A_i$$

Full Adder

Full adder made of several half adders

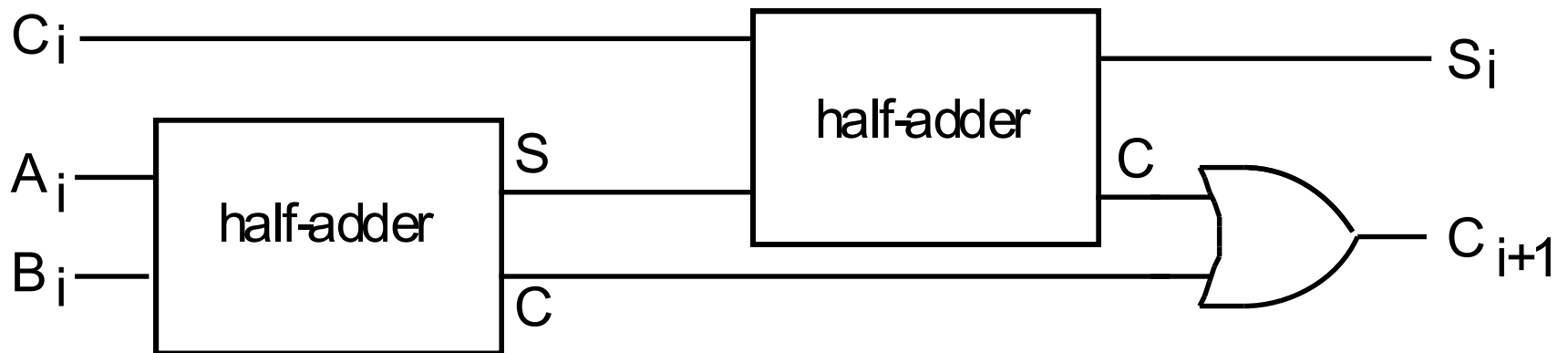
$$S_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

$$C_{i+1} = A_i B_i + C_i B_i + C_i A_i$$



Full Adder

Hardware repetition simplifies hardware design



A full adder can be made from two half adders (plus an OR gate).

Full adder equations

- Using Boolean algebra, we can simplify S and C_{out} as shown here.

X	Y	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{aligned} S &= \Sigma m(1,2,4,7) \\ &= X'Y'C_{in} + X'YC_{in}' + XY'C_{in}' + XYC_{in} \\ &= X'(Y'C_{in} + YC_{in}') + X(Y'C_{in}' + YC_{in}) \\ &= X'(Y \oplus C_{in}) + X(Y \oplus C_{in})' \\ &= X \oplus Y \oplus C_{in} \end{aligned}$$

$$\begin{aligned} C_{out} &= \Sigma m(3,5,6,7) \\ &= X'YC_{in} + XY'C_{in} + XYC_{in}' + XYC_{in} \\ &= (X'Y + XY')C_{in} + XY(C_{in}' + C_{in}) \\ &= (X \oplus Y)C_{in} + XY \end{aligned}$$

- Notice that XOR operations simplify things a bit, but we had to resort to using algebra since it's hard to find XOR-based expressions with K-maps.

The N -Bit Adder

- The next leap of faith is to move from adding 3 (or more) single digit numbers to adding multi-bit numbers
- First, note that the sum of adding two N -bit numbers can be $N + 1$ bits
- This comes as a result of possibly obtaining a carry into the next column
- Adding N -bit numbers isn't an abstract consideration – it's a **real** computing operation - at a minimum, a modern computer would add 16 bit numbers, done with a 16-bit adder

Multiple-bit Addition

Consider single-bit adder for each bit position.

$$\begin{array}{rcccc} & A_3 & A_2 & A_1 & A_0 \\ \mathbf{A} & 0 & 1 & 0 & 1 \end{array}$$

$$\begin{array}{rcccc} & B_3 & B_2 & B_1 & B_0 \\ \mathbf{B} & 0 & 1 & 1 & 1 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & \\ \mathbf{A} & 0 & 1 & 0 & 1 \\ \mathbf{B} & 0 & 1 & 1 & 1 \\ \hline & 1 & 1 & 0 & 0 \end{array}$$

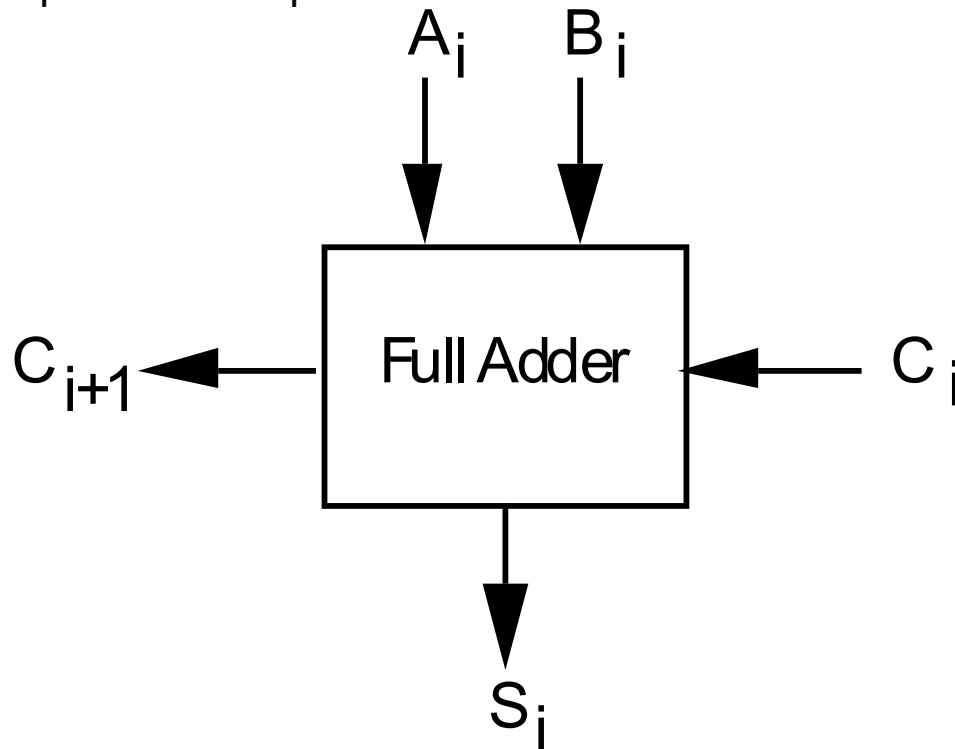
$$\begin{array}{r} C_{i+1} \leftarrow \\ C_i \\ A_i \\ + B_i \\ \hline S_i \end{array}$$

Each bit position creates a sum and carry

Full Adder

Putting it all together

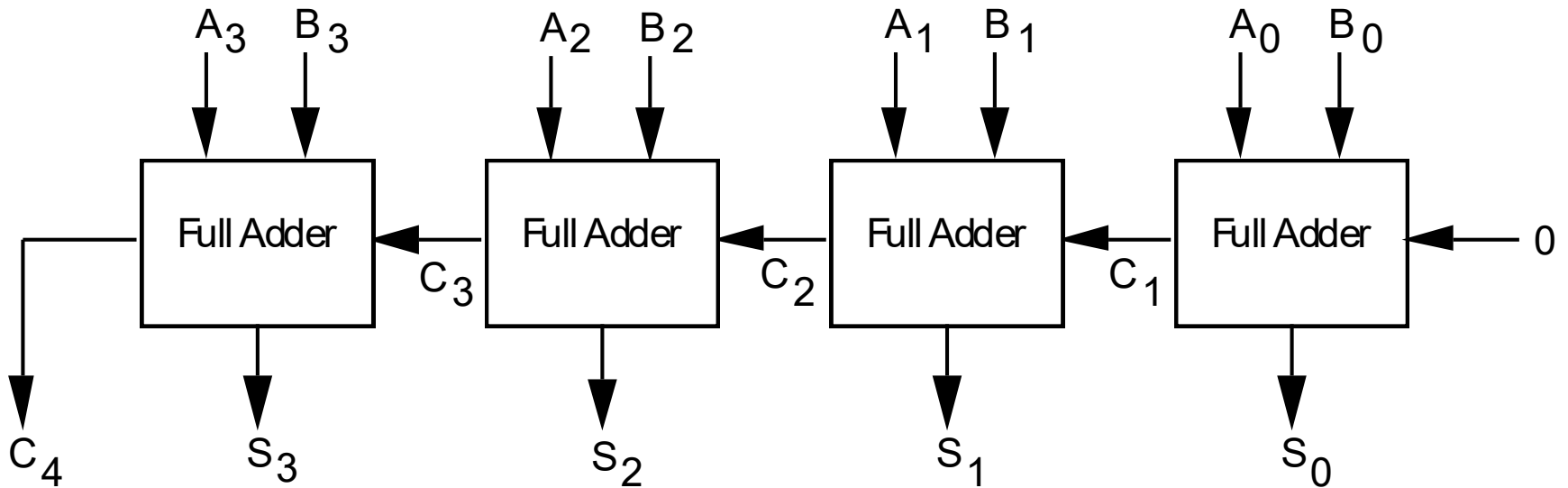
- Single-bit full adder
- Common piece of computer hardware



Block Diagram

4-Bit Adder

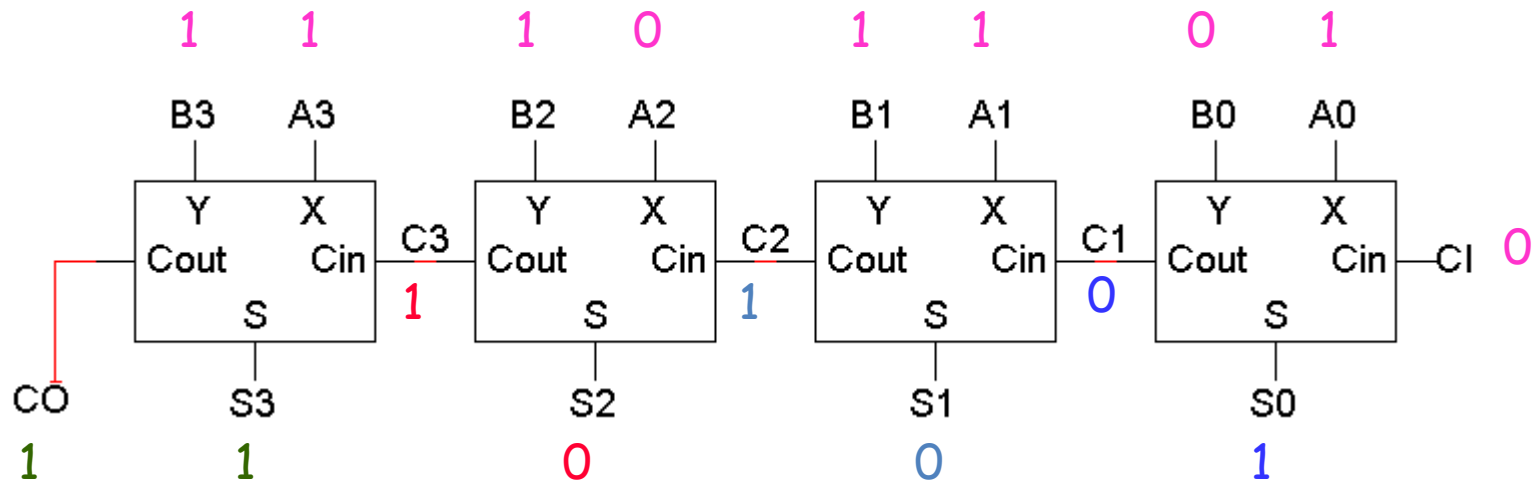
Chain single-bit adders together.
What does this do to delay?



C	1	1	1	0
A	0	1	0	1
B	0	1	1	1
S	1	1	0	0

An example of 4-bit addition

Let's try our initial example: $A=1011$ (eleven), $B=1110$ (fourteen).

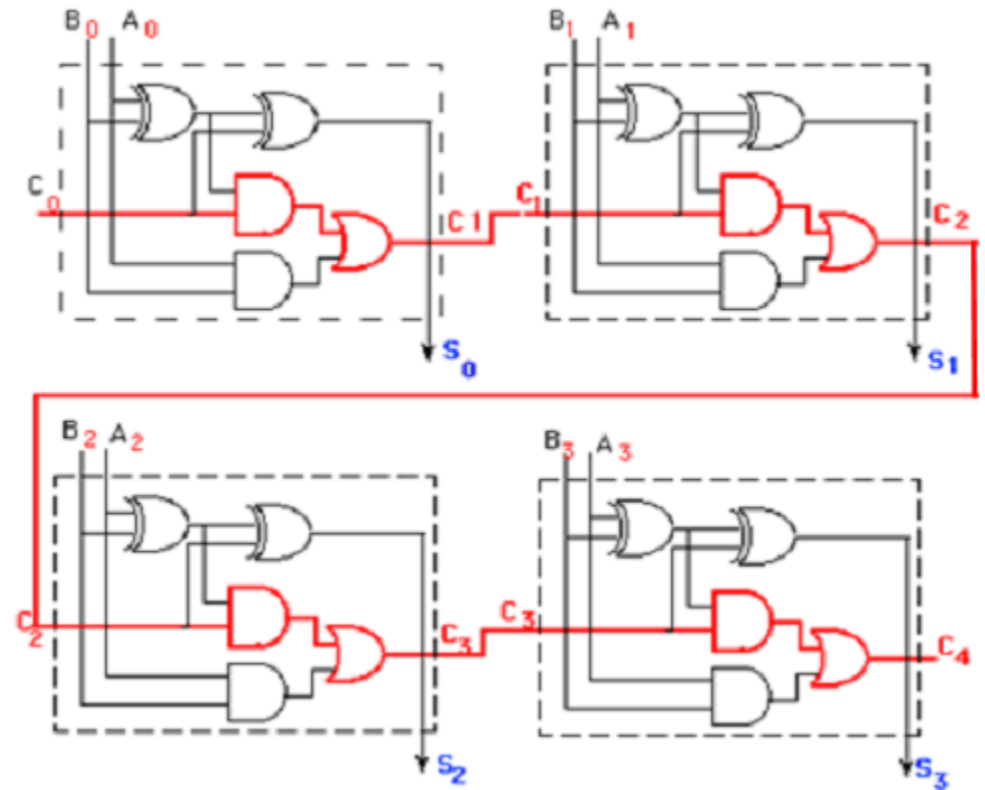


1. Fill in all the inputs, including $CI=0$
2. The circuit produces $C1$ and $S0$ ($1 + 0 + 0 = 01$)
3. Use $C1$ to find $C2$ and $S1$ ($1 + 1 + 0 = 10$)
4. Use $C2$ to compute $C3$ and $S2$ ($0 + 1 + 1 = 10$)
5. Use $C3$ to compute CO and $S3$ ($1 + 1 + 1 = 11$)

Ripple Carry Adder (RCA)

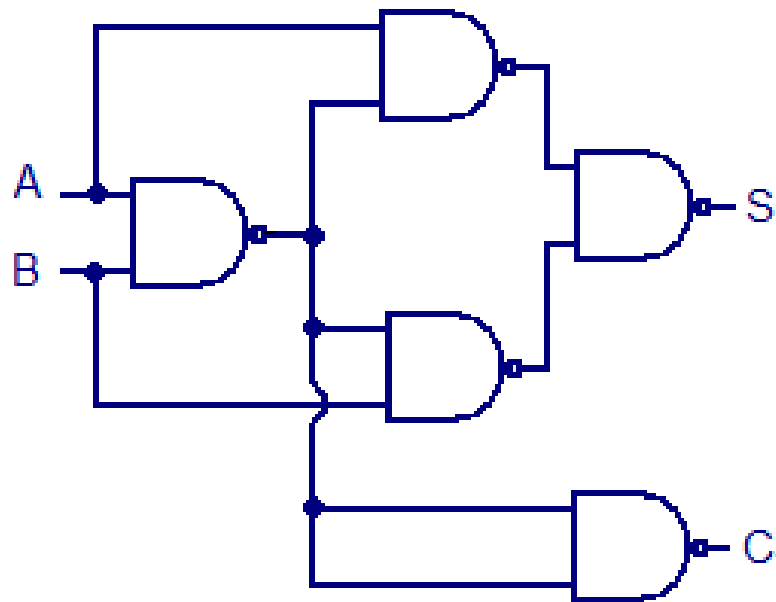
- Assume gate delay = T
- $8T$ to compute the last carry
- Total delay = $8 + 1 = 9T$
- 1 delay from first half adder
- Delay = $(2n+1)T$
- Where n = # of bit

How to improve?

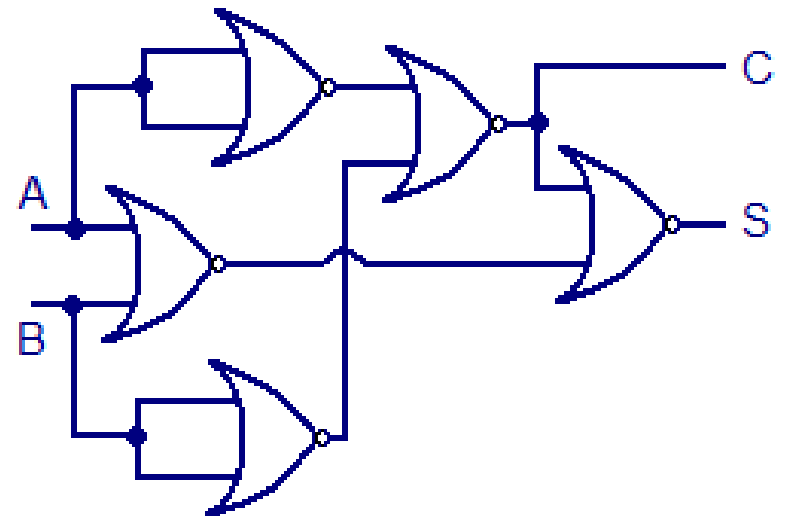


Carry Look Ahead Adder

- How to reduce propagation delay of ripple carry adders?
- **Carry look ahead adder:** All carries are computed as a function of C_0 (independent of n !)
- It works on the following standard principles:
 - A carry bit is generated when both input bits A_i and B_i are 1, or
 - When one of input bits is 1, and a carry in bit exists

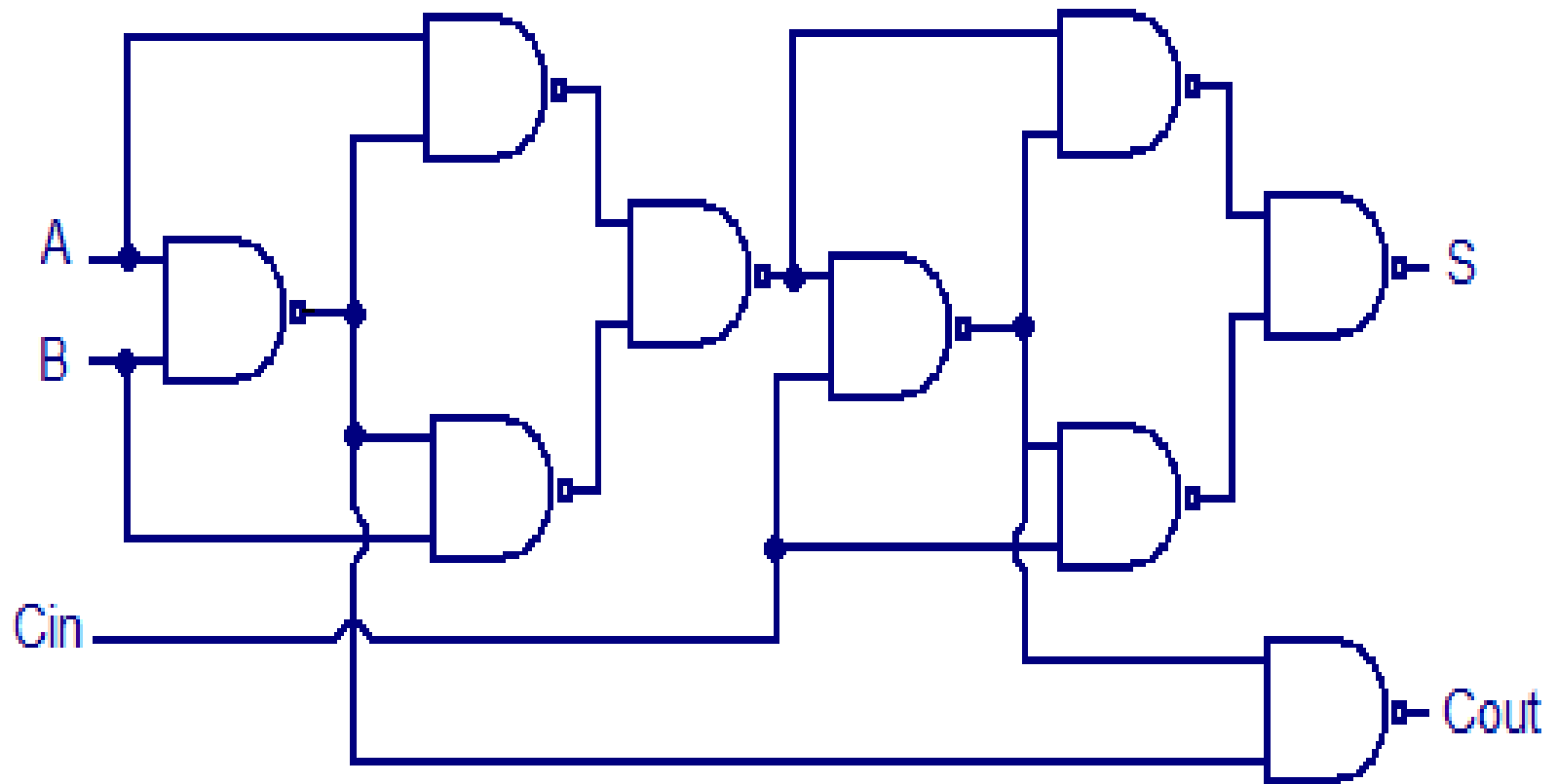


Half adder using NAND logic



Half adder using NOR logic

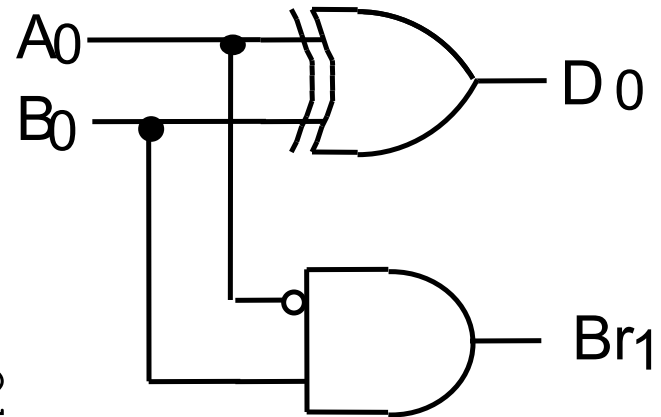
Full adder using NAND logic



Half Subtractor

A_0	B_0	D_0	Br_1
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$\begin{array}{r} 12 \\ 0 \\ -1 \\ \hline 1 \end{array}$$



Full Subtractor

A	B	C	D	BOR
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

		AB			
		00	01	11	10
C	0	0	1	0	1
	1	1	0	1	0

D

$$D = A \text{ XOR } B \text{ XOR } C$$

Same as Sum in full adder

Full Subtractor

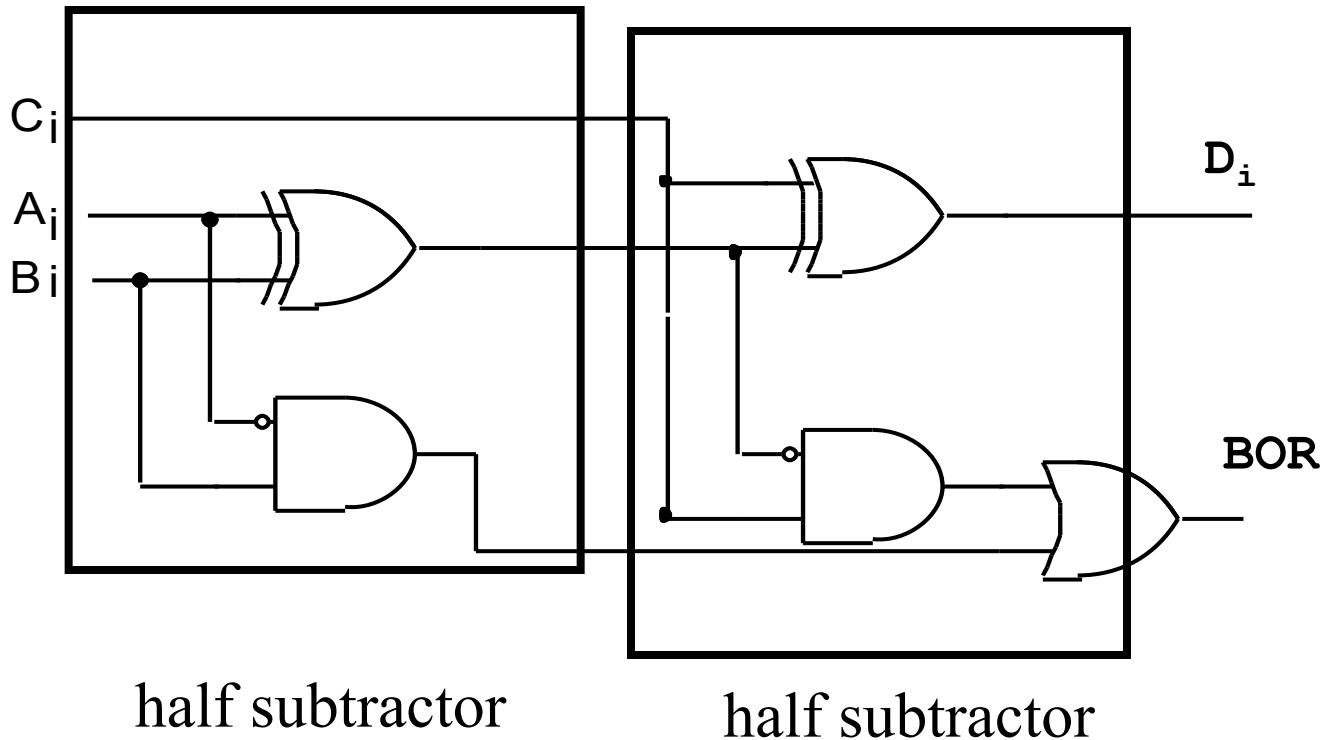
A	B	C	D	BOR
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$\text{BOR} = A' B + CB + CA'$$

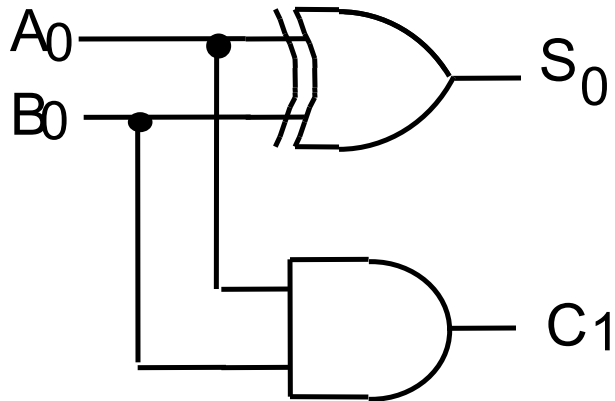
Full Subtractor

$$D_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

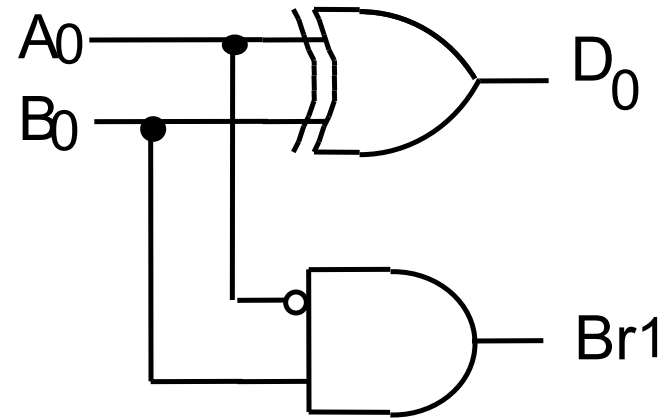
$$\text{BOR} = A_i' B_i + C_i B_i + C_i A_i'$$



Adder/Subtractor - 1



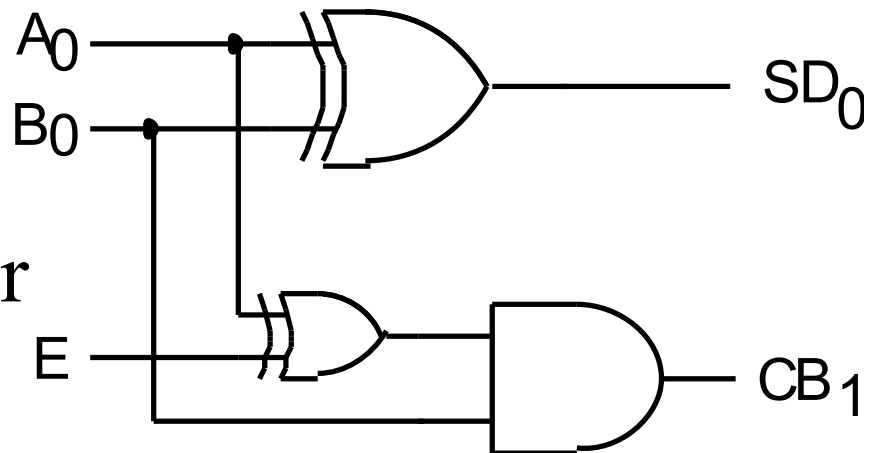
Half adder



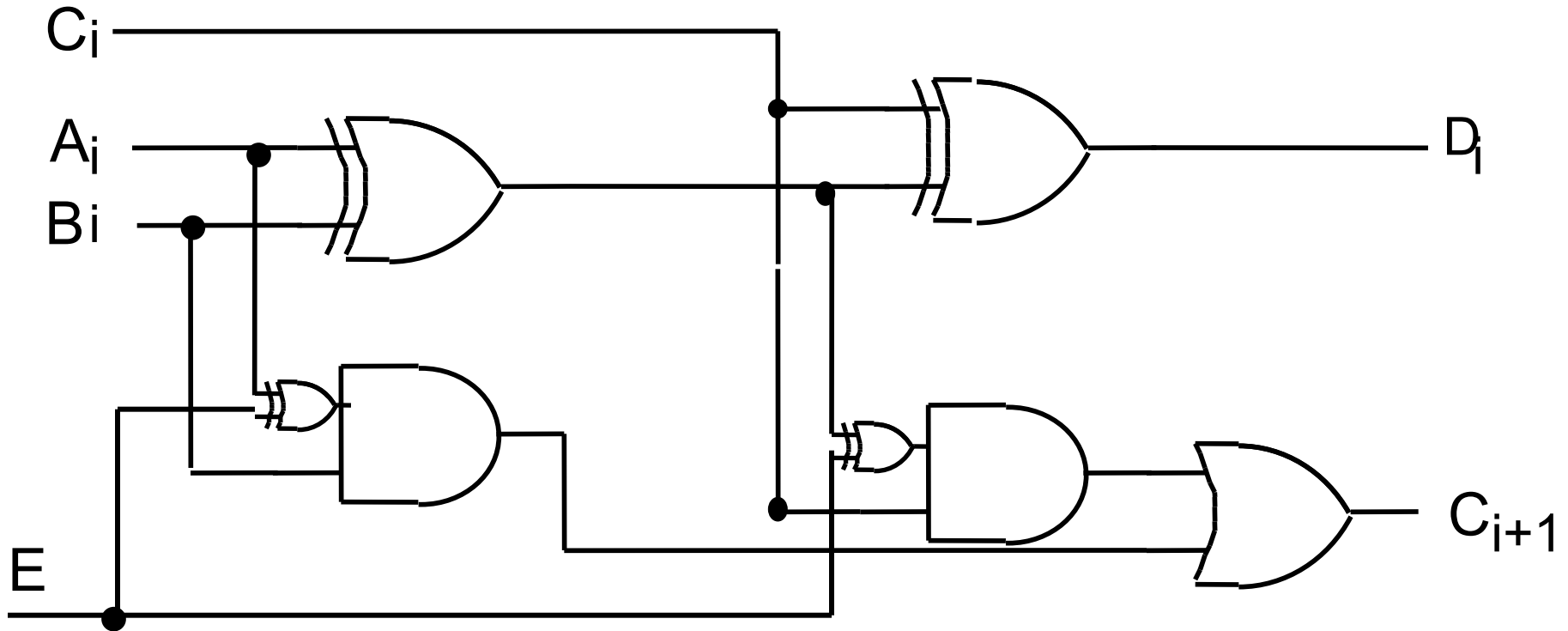
Half subtractor

$E = 0$: Half adder

$E = 1$: Half subtractor



Adder/Subtractor-1



$E = 0$: Full adder

$E = 1$: Full subtractor